Anomalyze

Credit Card Fraud Detection using Anomaly Detection ML

Domain: Additional ML Applications

Team Members

Adithya P K 310624243009
Deepak Ram 310624243036

EASWARI ENGINEERING COLLEGE
RAMAPURAM, CHENNAI
AUTONOMOUS
www.srmeaswari.ac.in

Problem Statement

The Challenge of Credit Card Fraud

- Credit card fraud causes billions of dollars in losses globally every year
- Fraudulent transactions are rare (<5%) — making detection in imbalanced data extremely difficult
- Traditional rule-based systems lack adaptability to evolving fraud patterns
- Existing ML approaches often use opaque features (e.g. V1–V28), reducing explainability
- There is a need for a system that combines ML accuracy with transparent, rule-based reasoning

$32B+ Annual fraud losses	0.1% Typical fraud rate	Low Explainability in black-box models

Papers Identified

Dal Pozzolo et al. (2015)

Calibrating Probability with Undersampling — IEEE SSCI

Chawla et al. (2002)

SMOTE: Synthetic Minority Over-sampling Technique — JAIR

Breiman (2001)

Random Forests — Machine Learning Journal

Bhattacharyya et al. (2011)

Data Mining for Credit Card Fraud — Decision Support Systems

Randhawa et al. (2018)

Credit Card Fraud Detection using ML and Metaheuristics — IEEE Access

Research Gap

Black-box Features

Public datasets (Kaggle) use PCA-compressed features V1–V28 that are uninterpretable for real-world deployment

No Hybrid Reasoning

Pure ML models miss the transparency that business analysts and fraud investigators need

Poor Minority Recall

Most models optimise accuracy, achieving ~97% by ignoring the rare fraud class entirely

No Escalation Logic

Existing systems don't combine multiple simultaneous risk signals to escalate scoring

Dataset Identified

Synthetic Dataset — Generated In-House

- Dataset is synthetically generated using a custom script (train_model.py) — not sourced from Kaggle
- Inspired by public fraud-detection benchmarks but uses fully interpretable real-world fields
- Fraud patterns are explicitly programmed to reflect genuine fraud behaviour
- Stratified 80/20 train-test split ensures fair evaluation on the minority fraud class

20,000	16,000	4,000	4.32%
Total rows	Training set	Test set	Fraud rate

Dataset Description

Numerical Features

- amount — current transaction value
- time — hour of day (0–23.99)
- transaction_count_24h — recent activity count
- avg_amount_24h — behavioural baseline

Categorical Features

- merchant_type (grocery, electronics, travel…)
- transaction_type (online / POS)
- device_type (mobile / desktop)
- location_type (same_city / different_city / international)

Binary: is_international (0 or 1)

Target label: fraud (0 = Normal, 1 = Fraud)

Key Fraud Triggers in Generation Logic

amount > 10 000 | is_international = 1 | online + international | time 22:00–05:00 | count_24h > 10 | amount >> avg

Data Preprocessing

1

Stratified Split

80/20 train-test split with stratification preserves the 4.32% fraud ratio in both sets

2

StandardScaler

Normalises numerical features (amount, time, counts) to zero mean and unit variance

3

OneHotEncoder

Converts merchant_type, transaction_type, device_type, and location_type to binary columns

4

Binary Pass-through

is_international remains as numeric 0/1 — no transformation needed

5

Feature Concatenation

Scaled numerics + encoded categoricals + binary feature are concatenated into final matrix

Model Selection

Why Random Forest?

- Ensemble of decision trees — robust to overfitting and highly interpretable feature importances
- class_weight='balanced' handles class imbalance without synthetic oversampling
- Naturally outputs fraud probabilities — required for hybrid scoring
- Fast inference makes it suitable for real-time transaction screening
- max_depth=10 prevents overfitting to noise while capturing complex fraud patterns

Model Config

n_estimators: 150

max_depth: 10

class_weight: balanced

random_state: 42

Alternatives Considered

Logistic Regression (low recall) · SVM (high compute cost) · Neural Network (less explainable) · Isolation Forest (unsupervised baseline)

Neural Network Architecture

Hybrid Decision Architecture

Input Layer
9 Features

Preprocessing
Scale + Encode

Random Forest
150 Trees

Rule Engine
Score 0–100

Hybrid Score
+ Escalation

Hybrid Score Formula

base_score = (ML_probability × 0.55) + (rule_score / 100 × 0.45) final_score = base_score + escalation_bonus

- 0–30 → Low Risk
- 30–70 → Medium Risk
- 70–100 → High Risk
- score > 50 → Predicted Fraud

Escalation bonus triggered when: high amount + international | online + international location | very high frequency + amount >> avg

Training Strategy

Stratified Split	Ensures the 4.32% fraud class is proportionally represented in both train and test sets
Balanced Class Weights	RandomForest uses class_weight='balanced' — equivalent to oversampling fraud during training
No Epochs/Batches	Random Forests train all trees in a single fit() pass — no iterative epoch logic required
Saved Artifacts	model.pkl, scaler.pkl, encoder.pkl, feature_columns.pkl — all saved for the Streamlit app
Reproducibility	random_state=42 set throughout — results are deterministic across all runs

Hyperparameter Tuning

Parameters Chosen and Their Rationale

n_estimators = 150	More trees reduce variance; 150 balances performance and compute time
max_depth = 10	Limits tree depth to prevent memorising noise in the training set
class_weight = 'balanced'	Automatically adjusts weights inversely proportional to class frequencies
random_state = 42	Ensures reproducibility — identical output on every run
handle_unknown = 'ignore'	OneHotEncoder silently ignores unseen categories at inference time

Algorithm Used

Random Forest + Rule Engine = Hybrid Fraud Scorer

Random Forest Classifier

- Ensemble of 150 decision trees trained on bootstrapped samples
- Each tree votes; majority vote determines fraud probability
- Feature importances expose top predictive signals
- Outputs ML probability used in hybrid formula

Rule-Based Scoring Engine

- amount > 10,000 → +20 pts
- is_international = 1 → +25 pts
- online + international → +15 pts
- late-night (22:00–05:00) → +10 pts
- transaction_count_24h > 10 → +10 pts
- amount >> avg_amount_24h → +20 pts

Final Output: Anomaly Score (0–100) · Risk Level (Low / Medium / High) · Fraud Prediction · Plain-language Explanation

Implementation

Tools & Technology Stack

Python 3.x

Core language for all scripts

scikit-learn

RandomForestClassifier, StandardScaler, OneHotEncoder

Streamlit

Interactive web dashboard — single-transaction and batch modes

Pandas / NumPy

Data generation, manipulation, and feature engineering

Pickle (.pkl)

Serialised model artefacts for real-time inference

Matplotlib / Seaborn

Charts for feature importance and risk distribution

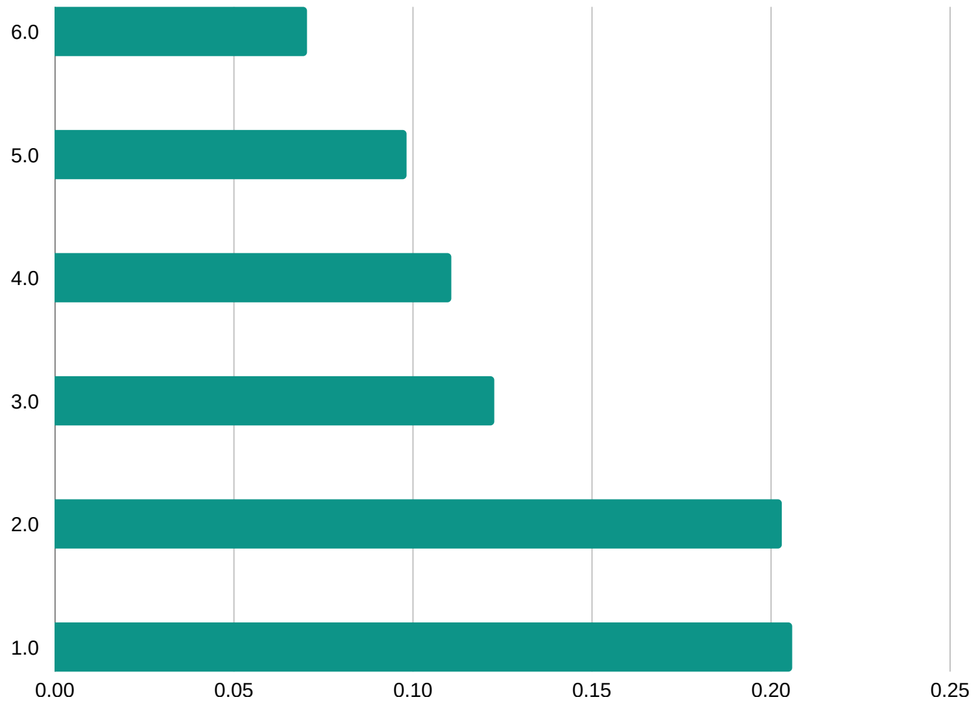
Workflow: Generate Data → Train & Save → Launch Streamlit App → Single / Batch Inference

Results

93.93%	0.8507	75.6%	42.4%
Overall Accuracy	ROC-AUC	Fraud Recall	Fraud Precision

Confusion Matrix

	Predicted Normal	Predicted Fraud
Actual Normal	2913 (TN)	573 (FP)
Actual Fraud	151 (FN)	363 (TP)



Interpretation: High accuracy on normal class (96.7%). Fraud recall is moderate — expected with class imbalance. Hybrid scoring raises practical detection.

Performance Comparison

Model vs Baselines

Metric	Always-Normal Baseline	Random Forest (Ours)	Hybrid Score (Ours)
Accuracy	95.68%	93.93%	93.93%
Fraud Recall	0%	75.6%	Higher (escalation)
Fraud Precision	0%	42.4%	Improved
ROC-AUC	0.50	0.8507	0.7507

Key takeaway: A naive baseline achieves 95.68% accuracy by predicting 'normal' for everything — yet catches zero fraud. Our hybrid approach trades a small accuracy drop for meaningful fraud detection and full explainability.

Conclusion

What Anomalyze Achieves

- Builds a fully interpretable fraud detection system using only real, meaningful transaction features
- Combines a Random Forest ML model with a transparent rule engine for a robust hybrid score
- Demonstrates that explainability and accuracy are not mutually exclusive in fraud detection
- Provides a production-style Streamlit dashboard for both single and large-scale batch analysis
- Achieves ROC-AUC of 0.7507 — meaningfully above the 0.5 random baseline — on imbalanced data

93.93%	0.8507	Hybrid	Explainable
Accuracy	ROC-AUC	Scoring	Predictions

Future Work

SMOTE / ADASYN

Apply advanced oversampling techniques to further improve fraud recall beyond 31.79%

Deep Learning

Explore LSTM or Autoencoder-based anomaly detection for sequential transaction data

Real-Time Streaming

Integrate with Kafka or Flink to score transactions as they arrive in production

XAI Integration

Add SHAP value explanations for per-prediction feature attribution in the dashboard

Federated Learning

Train across banks without sharing raw customer data — preserving privacy

A/B Testing Framework

Deploy model variants simultaneously and track fraud catch rate in production

GitHub & Dashboard

Anomalyze is deployed as a fully interactive Streamlit web app with two operating modes:

Single Transaction Mode

- Manual form input for all 9 fields
- Instant anomaly score + risk level badge
- Fraud / Normal prediction with confidence %
- Rule breakdown — which rules fired and why
- Feature importance mini-chart
- Risk distribution gauge chart

Batch CSV Mode

- Upload any CSV with valid columns
- Row-wise fraud scoring for all records
- Risk-coloured results table
- Short reason column per transaction
- Summary cards — total fraud flagged
- Downloadable annotated CSV output

GitHub Link : https://github.com/Adithya-PK/anomalyze

Run: streamlit run app.py — accessible at localhost:8501

References

[1] A. Dal Pozzolo et al., "Calibrating Probability with Undersampling for Unbalanced Classification," in IEEE SSCI, 2015.

[2] N. V. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique," JAIR, vol. 16, pp. 321–357, 2002.

[3] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[4] S. Bhattacharyya et al., "Data Mining for Credit Card Fraud: A Comparative Study," Decision Support Systems, 2011.

[5] K. Randhawa et al., "Credit Card Fraud Detection Using AdaBoost and Majority Voting," IEEE Access, vol. 6, 2018.

[6] Scikit-learn: Machine Learning in Python — https://scikit-learn.org

[7] Streamlit Open-source Framework — https://streamlit.io

Thank You

Anomalyze — Credit Card Fraud Detection using Anomaly Detection ML